

Codebase Doctor

Architecture and Phased Development Plan

Version 1 – Python-First Repository Analysis

Project Path: C:\Users\ADMIN\codebase-doctor

The project will be developed locally first using Git. The empty GitHub remote can be connected later.

Implementation Rule: Do not implement all phases in one pass. Each phase must be implemented only after explicit approval to begin that phase.

1 High-Level Architecture

1.1 Scope

The v1 implementation is Python-first.

Other file types may be cloned and counted, but deep bug, dead-code, security, and architectural analysis will primarily target Python.

The LLM layer is pluggable and OpenAI-compatible through:

```
LLM_BASE_URL
LLM_API_KEY
LLM_MODEL
```

1.2 Non-Goals for v1

The following are explicitly excluded from version 1:

- PostgreSQL
- Redis/Celery
- Neo4j
- Kubernetes
- Microservices
- Executing tests from scanned repositories
- Automatically applying patches
- Multi-language deep analysis

1.3 System Flow

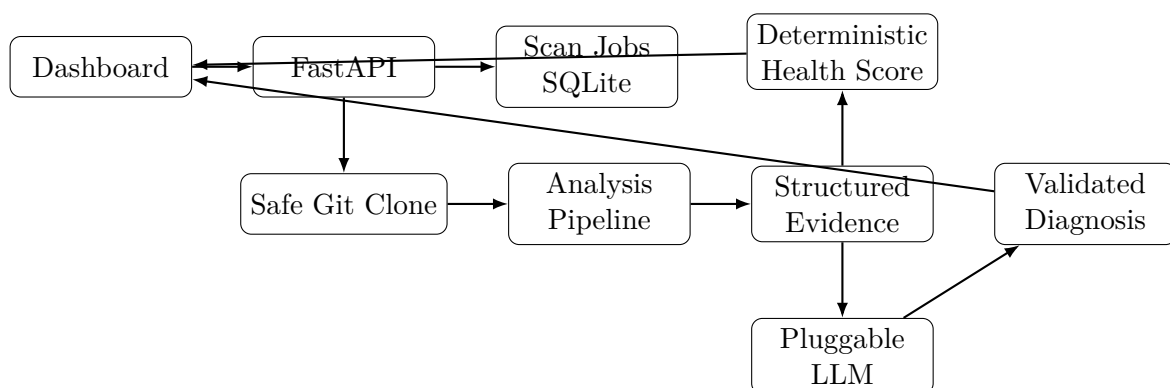


Figure 1: High-Level Codebase Doctor Architecture

1.4 Execution Flow

The complete workflow is:

1. User submits a GitHub repository URL.
2. Backend validates the URL.
3. Repository is shallow-cloned into an isolated workspace.
4. Clone size, file count, file size, and execution time are limited.
5. File inventory is generated.
6. Static analyzers run against the repository.
7. Analyzer output is normalized into structured findings.
8. Deterministic health scores are calculated.
9. Optional AI analysis consumes a capped evidence pack.
10. AI output is validated using Pydantic schemas.
11. Dashboard displays scores, findings, diagnosis, and proposed fixes.

The AI never writes to the cloned repository.

1.5 Persistence

SQLite with SQLAlchemy is used for persistence.

Persistence is justified because the application needs:

- Scan history
- Findings
- Health scores
- Architecture metrics
- AI outputs
- Patch proposals

An in-memory implementation would make demonstrations fragile.

1.6 Job Model

Redis and Celery are intentionally excluded.

Instead:

- A `scans` table stores job state.
- `asyncio.to_thread` or an executor runs scans in the background.
- One scan runs concurrently in v1.
- Additional scans remain in `queued` state.

2 Backend Architecture

The backend is implemented using FastAPI with a pipeline orchestrator and isolated analyzer adapters.

Layer	Responsibility
API	Academic, auth-less API. Validates inputs, starts and polls scans, fetches reports, requests AI analysis, and never applies patches.

Clone / Workspace	Performs shallow cloning using <code>git clone -depth 1</code> . Enforces size, file, timeout, symlink, and path-traversal protections.
Analyzers	Inventory, Ruff, Bandit, Semgrep, pip-audit, AST/Tree-sitter architecture analysis, and custom Python heuristics.
Evidence Store	Normalizes all analyzer results into common Finding records.
Scoring	Pure deterministic functions over findings and architecture metrics.
AI	Builds an evidence pack, calls the LLM, validates or repairs JSON, and persists AI proposals.
Security	GitHub URL allowlisting, SSRF checks, redaction, and prompt-injection hardening.

2.1 Subprocess Policy

Analyzer processes follow strict subprocess rules:

- Use fixed tool binaries from our virtual environment.
- Set `cwd` to the cloned repository root.
- Use explicit timeouts.
- Never use shell execution.
- Strip unnecessary environment variables.
- Capture stdout and stderr.
- Truncate captured output.

3 Frontend Architecture

The frontend uses:

- Vite
- React
- TypeScript
- Tailwind CSS

Route	Purpose
/	Submit GitHub URL, configure limits, and start scan.
/scans/:id	Live scan status using polling.
/scans/:id/dashboard	Health scores, charts, and finding counts.
/scans/:id/findings	Filter findings by category, severity, and analyzer.
/scans/:id/architecture	Import graph summary, module relationships, and circular dependencies.
/scans/:id/diagnosis	AI-generated diagnosis cards.
/scans/:id/fixes	Monaco-based original/proposed code comparison and unified diff.

3.1 Frontend Safety

There must be no fake charts.

Charts remain empty until a scan has completed successfully.

Finding messages and code snippets must be:

- Escaped
- Length-capped

- Secret-redacted
- Rendered as text

React's default escaping is used and `dangerouslySetInnerHTML` is prohibited.

The patch page contains an **Apply** button only as a disabled control with the explanation:

Export patch only

4 Repository Analysis Pipeline

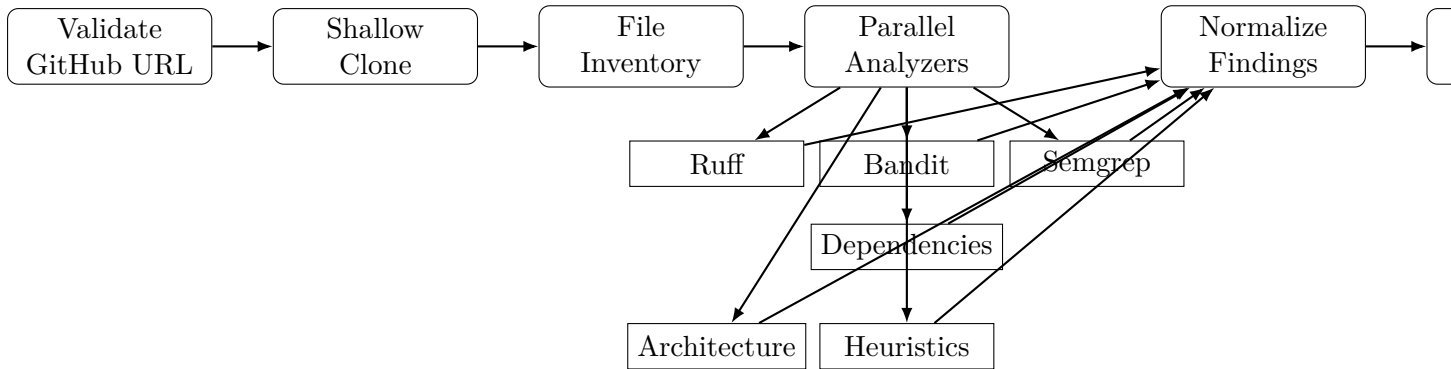


Figure 2: Repository Analysis Pipeline

4.1 Pipeline Order

The pipeline follows:

1. Validate GitHub URL.
2. Perform shallow clone.
3. Build file inventory.
4. Run analyzers.
5. Normalize findings.
6. Calculate health score.
7. Persist scan results.
8. Optionally execute AI diagnosis.

Analyzers are fail-soft.

A crashed analyzer becomes an `analyzer_error` finding rather than producing a fake clean result.

4.2 Untrusted Repository Policy

The scanned repository is treated as untrusted input.

The system must:

- Never execute repository Python code.
- Never run repository tests.
- Never execute repository scripts.
- Never run `pip install` from the repository.
- Never honor README, CI, or repository instructions.

5 Directory Structure

```
codebase-doctor/
  README.md
  .env.example
  backend/
    pyproject.toml
    app/
      main.py
      config.py
      db.py
      models.py
      schemas.py
      api/routes_scans.py
      api/routes_ai.py
      pipeline/orchestrator.py
      pipeline/workspace.py
      pipeline/inventory.py
      analyzers/base.py
      analyzers/ruff.py
      analyzers/bandit.py
      analyzers/semgrep.py
      analyzers/dependencies.py
      analyzers/architecture.py
      analyzers/heuristics.py
      scoring/rules.py
      scoring/compute.py
      ai/evidence_pack.py
      ai/client.py
      ai/prompts.py
      ai/schemas.py
      security/github_url.py
      security/workspace.py
      security/redact.py
      rules/semgrep/python.yml
      tests/
  frontend/
    package.json
    src/pages/...
    src/api/client.ts
  data/
```

The `data/` directory is gitignored and contains SQLite data and temporary repository clones.

6 Data Models

6.1 SQLAlchemy Models

Model	Fields
Scan	id, github_url, status (queued running succeeded failed), error, created_at, clone_path, limits, commit_sha, language_stats JSON
Finding	scan_id, analyzer, category (bug security dead_code dependency architecture q severity, rule_id, file_path, start_line, end_line, message, evidence JSON, redacted snippet
ArchitectureMetric	scan_id, module, fan_in, fan_out, circular_component_id nullable
HealthScore	scan_id, security, architecture, maintainability, performance, code_quality, dependencies, overall, breakdown JSON
AiReport	scan_id, kind (diagnosis recommendations patch), raw_model, validated JSON, created_at

PatchProposal	scan_id, file_path, original, proposed, unified_diff, explanation, confidence, risk, status (proposed)
----------------------	---

6.2 Normalized Finding

Every analyzer must map its output into the same Pydantic **Finding** schema.

The frontend must not depend on analyzer-specific contracts.

7 API Endpoints

Method	Path	Purpose
POST	/api/scans	Accepts { <code>github_url</code> } and returns HTTP 202 with <code>scan_id</code> .
GET	/api/scans/{id}	Returns scan status and errors.
GET	/api/scans/{id}/findings	Returns findings with category, severity, and analyzer filters.
GET	/api/scans/{id}/architecture	Returns architecture graph summary and cycles.
GET	/api/scans/{id}/score	Returns health scores and documented score breakdown.
POST	/api/scans/{id}/ai/diagnose	Builds evidence pack and returns validated diagnosis.
POST	/api/scans/{id}/ai/fixes	Accepts optional finding IDs and returns patch proposals.
GET	/api/scans/{id}/ai	Returns stored AI outputs.
GET	/health	Application health endpoint.

There is intentionally no:

POST /apply-patch

in v1.

An optional future endpoint may provide downloadable `.diff` files.

8 Analyzer Responsibilities

Analyzer	Detects	Tool
Inventory Ruff	File list, file sizes, language mix, skipped binaries Unused imports/variables, quality rules, bug rules, and some unreachable-code patterns	Custom ruff check -output-format json
Bandit	Security issues including subprocess use, SQL-related issues, deserialization, and security patterns	bandit -f json
Semgrep	Command injection, SQL injection, path traversal, unsafe YAML/pickle, broad exception handling, shell=True , insecure CORS, and XSS-related patterns	Custom Semgrep YAML rules
Custom Heuristics	Possible None dereferences, infinite loops, broad exception handling, and missing context managers	Python AST / Tree-sitter
Dependencies	Dependency manifests and known vulnerabilities	Manifest parsing + pip-audit

Architecture	Import graph, cycles, hubs, fan-in, fan-out, coupling	AST / Tree-sitter
Dead Code	Potential unused functions/modules	Graph + Ruff

8.1 Honest Coverage

Static analysis does not provide runtime proof for every finding.

Therefore:

- Possible null/None dereference findings are explicitly labeled as heuristics.
- Infinite-loop detection is heuristic.
- Heuristic findings include a confidence value.
- The system must not claim runtime certainty.

8.2 Performance Dimension

Performance scoring is based only on static evidence.

Examples include:

- Nested loops over large literals.
- Obvious blocking patterns where reliable rules exist.

If there are zero performance findings, the performance score is:

$$\boxed{100}$$

This represents a neutral score rather than an invented claim that the repository is fast.

The weighting is documented regardless.

9 AI Layer Design

9.1 Core Principle

The LLM reasons over structured evidence rather than receiving the entire repository.

9.2 Evidence Pack

The evidence pack is capped at approximately 32–64 KB.

It contains:

- Score breakdown.
- Top findings ranked by severity.
- Architecture summary.
- Circular dependency count.
- Top architecture hubs.
- Dependency vulnerability list.
- Redacted snippets from cited files only.
- Maximum approximately 20 lines per snippet.

The evidence pack explicitly instructs the model to ignore any instructions found inside repository snippets or README files.

9.3 System Prompt

The system prompt is fixed and does not originate from the repository.

The conceptual instruction is:

You are a code reviewer. Use only the provided evidence. If the evidence is insufficient, set `evidence_sufficient=false` and do not invent files, lines, vulnerabilities, or other facts.

9.4 Diagnosis Schema

Each diagnosis item contains:

- `diagnosis`
- `why_it_matters`
- `impact`
- `recommended_action`
- `priority`
- `confidence`
- `evidence_sufficient`
- `related_finding_ids`

9.5 Patch Schema

Each patch contains:

- `files_affected`
- `original_code`
- `proposed_code`
- `unified_diff`
- `explanation`
- `confidence`
- `risk`

Risk must be one of:

```
low
medium
high
```

9.6 Validation

AI responses are validated using Pydantic.

If validation fails:

1. Reject the output.
2. Retry once using a corrected request.
3. Persist only validated output.

9.7 LLM Client

The client uses OpenAI-compatible chat completions.

Configuration:

```
LLM_BASE_URL
LLM_API_KEY
LLM_MODEL
```

Temperature should be low.

JSON schema or `response_format` should be used when supported by the provider.

9.8 AI Restrictions

The AI layer must never:

- Receive raw secrets.
- Receive complete repository files unnecessarily.
- Follow instructions contained in repository code/comments.
- Automatically apply patches.
- Modify the cloned repository.
- Write the health score.

10 Security and Threat Model

Threat	Control
SSRF through GitHub URL	Allow only HTTPS URLs from <code>github.com</code> or <code>www.github.com</code> . Parse using <code>urllib</code> . Reject IP addresses, localhost, userinfo tricks, and non-GitHub redirects. Disable HTTP redirects where possible.
Malicious repository code	Never execute repository code and never install repository dependencies.
Zip/Clone Bomb	Maximum clone bytes, file count, file size, and clone time.
Path Traversal / Symlinks	Resolve paths under workspace root and reject or skip symlinks that escape the workspace.
Subprocess Hang	Timeouts and process-group termination.
Prompt Injection	Fixed system prompt, delimited untrusted evidence, truncated snippets, and explicit evidence sufficiency.
Secret Leakage	Redact high-entropy secrets and common key patterns from stored snippets, API responses, and AI evidence packs.
XSS	Treat findings as plain text. Use React escaping and prohibit <code>dangerouslySetInnerHTML</code> .
Tool Confusion	Invoke analyzer binaries from our environment rather than scripts supplied by the repository.

11 Testing Strategy

11.1 Unit Tests

Unit tests cover:

- GitHub URL validation and SSRF cases.
- Secret redaction.
- Scoring formulas.
- Golden-number score calculations.
- Evidence-pack size limits.
- Pydantic AI schemas.

11.2 Analyzer Adapter Tests

Analyzer adapters receive mocked subprocess output.

Fixtures should contain realistic JSON output from:

- Ruff
- Bandit
- Semgrep

The adapter tests verify conversion into normalized `Finding` records.

11.3 Pipeline Tests

A small fixture repository in `tests/` should contain known issues such as:

- Unused imports.
- `shell=True`.
- Circular imports.

The pipeline tests assert expected findings and scores.

11.4 API Tests

FastAPI `TestClient` tests should cover:

- Scan creation.
- Scan polling.
- Error handling.

The workspace layer should be mocked.

11.5 AI Tests

The AI layer should test:

- Mock HTTP responses.
- Invalid JSON.
- Validation retry.
- Prompt-injection snippets.
- Prevention of invented CVEs or file locations.

No test may execute code from a cloned third-party repository.

12 Development Phases

The project is implemented one phase at a time.

12.1 Phase 0 – Scaffold

Create:

```
C:\Users\ADMIN\codebase-doctor
```

Tasks:

- Initialize Git.
- Move the project agent/configuration to the root if required.
- Create FastAPI application.

- Create Vite + React application.
- Configure SQLite.
- Create `.env.example`.
- Create README.
- Add scoring rules stub.

12.2 Phase 1 – Safe Clone and Inventory

Implement:

- GitHub URL validation.
- Safe shallow cloning.
- Clone limits.
- File inventory.
- Scan status persistence.
- Inventory API.

Demo goal:

Submit a public Python repository and display file counts.

12.3 Phase 2 – Deterministic Analyzers

Implement:

- Ruff.
- Bandit.
- Semgrep.
- pip-audit.
- Architecture graph.
- Custom heuristics.
- Finding persistence.
- Findings UI.

12.4 Phase 3 – Health Scores and Dashboard

Implement:

- Documented score weights.
- Deterministic score calculation.
- Score persistence.
- Score breakdown.
- Dashboard.
- Recharts visualizations based only on real scan data.

12.5 Phase 4 – AI Diagnosis

Implement:

- Evidence pack generation.

- Pluggable LLM client.
- Prompt templates.
- Pydantic validation.
- Diagnosis endpoint.
- Diagnosis UI.

12.6 Phase 5 – Patch Proposals

Implement:

- Patch proposal schema.
- Monaco editor.
- Original vs proposed code.
- Unified diff viewer.
- Patch persistence.
- Export-only workflow.

Automatic patch application remains disabled.

12.7 Phase 6 – Hardening

Implement:

- Secret redaction.
- Prompt-injection tests.
- Clone and file limits.
- Analyzer timeout hardening.
- Security regression tests.
- README demo instructions.

13 Health Score

13.1 Overall Formula

The overall health score is calculated deterministically:

$$\begin{aligned} \text{Overall} = & 0.25(\text{Security}) + 0.15(\text{Architecture}) \\ & + 0.15(\text{Maintainability}) + 0.10(\text{Performance}) \\ & + 0.20(\text{CodeQuality}) + 0.15(\text{Dependencies}) \end{aligned}$$

The LLM has no control over this calculation.

13.2 Dimension Scoring

Every dimension begins with:

$$\text{Score} = 100$$

Penalties are then subtracted based on findings.

Example severity penalties:

Severity	Penalty
Critical	−15
High	−8
Medium	−3
Low	−1

Scores have a minimum floor of:

$$0$$

Therefore:

$$Score = \max(0, 100 - \text{penalties})$$

13.3 Architecture Penalties

Architecture scoring additionally considers:

- Circular dependency count.
- Hub concentration.
- Fan-in.
- Fan-out.
- Coupling.

13.4 Performance Neutral Score

If there is no static performance evidence:

$$Performance = 100$$

This is explicitly documented as a neutral value and does not imply that the repository has been proven performant.

13.5 Auditable Breakdown

The database stores a JSON breakdown containing:

- Dimension.
- Starting score.
- Every finding penalty.
- Architecture penalties.
- Weights.
- Final dimension score.
- Overall calculation.

A reviewer must be able to recompute the final score manually.

14 Implementation Principles

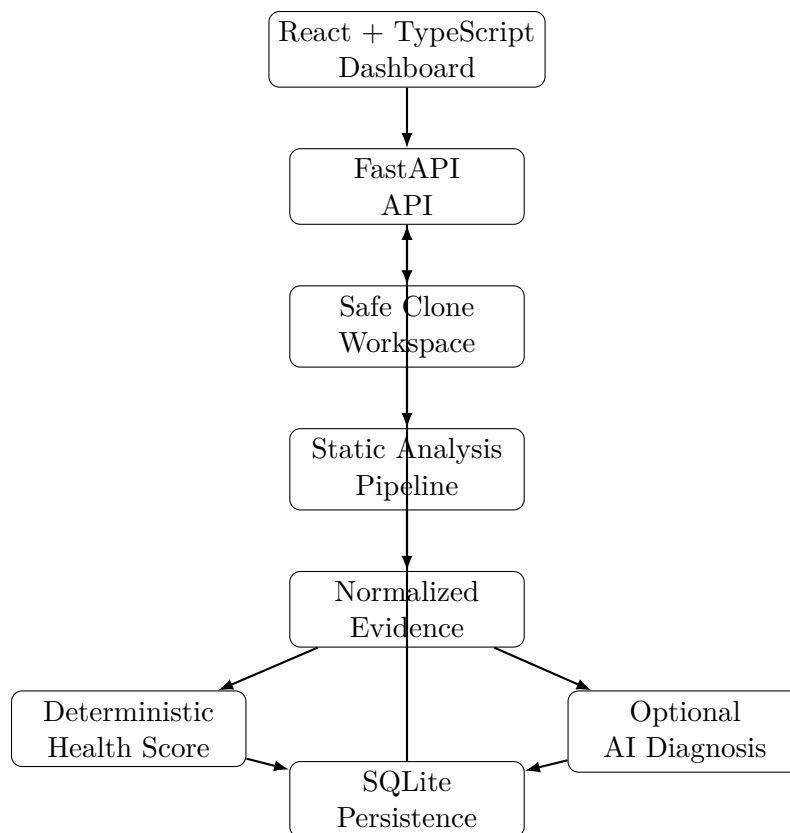
The following principles are mandatory for v1:

1. **Evidence before AI.**

2. **Deterministic scoring.**
3. **No execution of untrusted repository code.**
4. **No automatic patch application.**
5. **Fail-soft analyzers.**
6. **Normalized findings.**
7. **Capped and redacted AI evidence.**
8. **Auditable health-score breakdowns.**
9. **Honest labeling of heuristic findings.**
10. **One development phase at a time.**

15 Final v1 Architecture Summary

The final v1 system follows this architecture:



15.1 Implementation Boundary

The project should not proceed beyond the currently approved phase.

Implementation must begin only when an explicit instruction such as:

“Implement Phase 0”

is provided.

No later phase should be silently implemented in advance.